INSTITUT SUPÉRIEUR D'INFORMATIQUE ET DE MULTIMÉDIA DE GABÈS
Département Informatique Industrielle & Systèmes Embarqués
Licence en Informatique des Systèmes Industriels — LISI 3

COMPTE RENDU DE TP N° 2

# Prise en Main des GPIOs en Mode Registre (Bare-Metal) & Environnement Keil µVision sur STM32F407VGT

Création de projet sous Keil µVision, analyse ligne par ligne des registres matériels (RCC & GPIO), résolution d'erreur de compilation, téléversement (LOAD sur carte via ST-LINK) et débogage par le curseur.

`STM32F407VGT6` `ARM Cortex-M4F` `Keil µVision 5` `Bare-Metal CMSIS` `ST-LINK / V2` `Broche PA5`

ÉQUIPE DE RÉALISATION — GROUPE B

- ✓ **Mohamed Amine Bouguila**
- ✓ **Anis**
- ✓ **Yousri**

DÉTAILS ACADÉMIQUES

**Établissement :** ISIM Gabès (Univ. de Gabès)
**Classe :** LISI 3 (Groupe B)
**Module :** Systèmes Embarqués (ARM Cortex-M4)
**Année :** 2026 - 2027

## I Objectifs Pédagogiques & Banc Expérimental Réel

Ce TP2 a pour but d'apprendre la programmation **Bare-Metal** (accès direct aux registres sans bibliothèque HAL) sur le microcontrôleur **STM32F407VGT6** (cœur ARM Cortex-M4).

**Objectifs de la séance :**

- Comprendre la hiérarchie des bus et l'activation d'horloge **Clock Gating** (registre `RCC->AHB1ENR`).
- Configurer le mode d'une broche en sortie générale (01) via `GPIOA->MODER` avec des masques binaires.
- Contrôler le niveau logique de la sortie en temps réel avec le registre `GPIOA->ODR`.
- Créer un projet propre sous **Keil µVision 5**, résoudre l'erreur de linker, flasher (**LOAD**) et tester le code avec le curseur.

### 1. Montage Électrique sur Breadboard

```
[STM32F4 PA5] ──(Fil Violet)──► [ Résistance R = 330 Ω ] ──► [ Anode (+) LED ] ──► [ Cathode (-) ] ──► [ GND STM32 (Fil Vert) ]
```

Calcul du courant de protection : $I_F = \frac{V_{DD} - V_F}{R} = \frac{3.3\text{ V} - 2.0\text{ V}}{330\,\Omega} \approx 3.94\text{ mA}$, ce qui protège l'étage Push-Pull de la broche PA5.

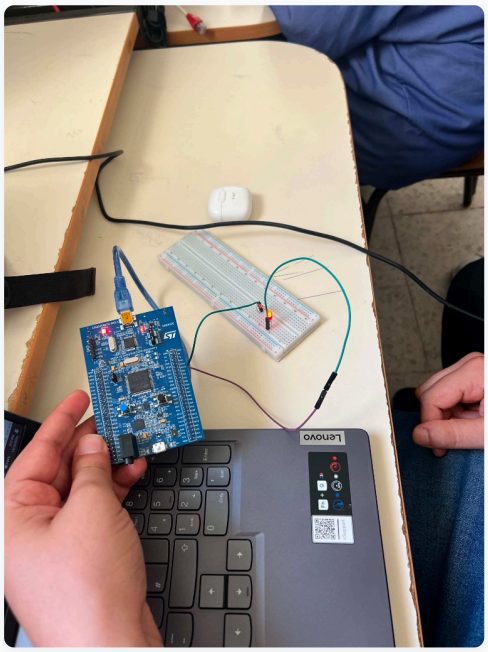



*Figure 1 : Banc d'essai réel du Groupe B (Carte STM32F407G-DISC1 connectée par câble ST-LINK, breadboard avec LED et résistance 330 Ω sur PA5 et GND).*

## II Création et Configuration du Projet dans Keil µVision

Avant d'écrire le code, la première étape pratique en laboratoire consiste à créer le projet dans Keil comme illustré ci-dessous avec les **flèches rouges** :

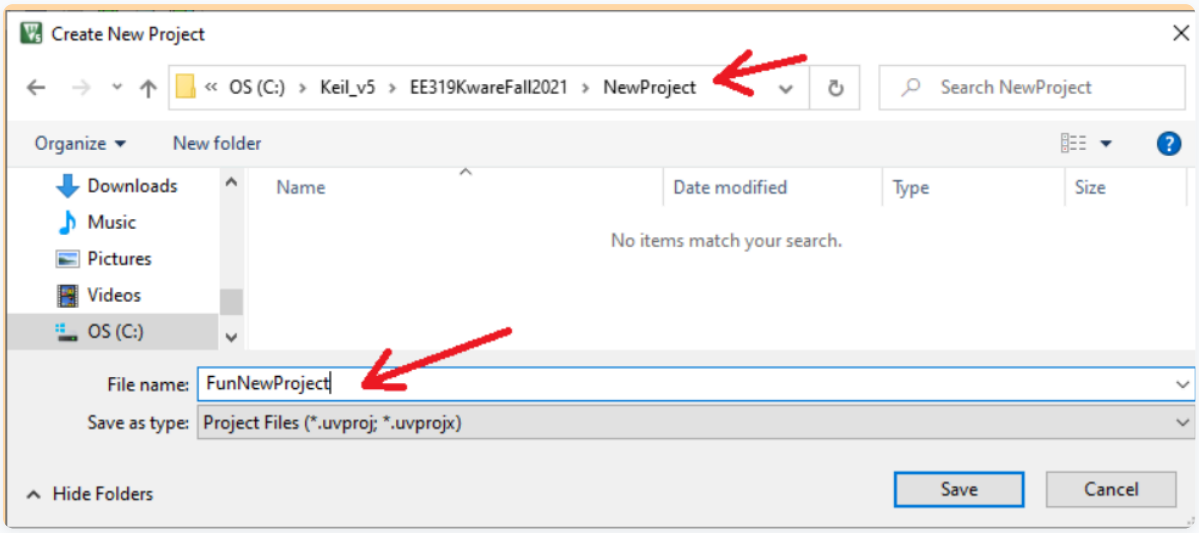



*Figure 2 : Boîte de dialogue "Create New Project" dans Keil µVision avec indication des flèches rouges (Sélection du dossier et Nom du fichier projet).*

1. **Étape 1 : Créer le Projet** FLÈCHE 1 & 2
   Dans la barre de menu de Keil, cliquer sur **Project** → **New µVision Project...**. Sélectionner le répertoire de travail (ex: `C:\...\TP2_STM32`) et saisir le nom du projet (ex: `tp_led`), puis cliquer sur **Save**.

2. **Étape 2 : Sélectionner le Microcontrôleur Cible**
   Dans la liste déroulante : choisir **STMicroelectronics** → **STM32F4 Series** → **STM32F407** → **STM32F407VGTx**, puis valider avec **OK**.

3. **Étape 3 : Configurer l'Environnement Runtime (RTE)**
   Dans la fenêtre contextuelle *Manage Run-Time Environment* :
   - Cocher **CMSIS** → **CORE**.
   - Cocher **Device** → **Startup** (ajoute le fichier assembleur de démarrage `startup_stm32f407xx.s`).
   - Valider en cliquant sur **OK**.

4. **Étape 4 : Ajouter le Fichier Source main.c**
   Dans l'arborescence du projet à gauche, faire un clic droit sur **Source Group 1** → **Add New Item to Group...** → **C File (.c)**, le nommer `main.c` pour y implémenter le code bare-metal.

## III Analyse Détaillée du Code C (Bloc par Bloc, 1 par 1)

Voici l'explication découpée bloc par bloc du code source implémenté dans `main.c` :

### Bloc 1 : Inclusion de l'En-tête CMSIS

```
#include "stm32f407xx.h"
```

**Explication :** Ce fichier définit les adresses physiques des périphériques (ex: `GPIOA = 0x40020000`, `RCC = 0x40023800`) et les structures de registres CMSIS (`GPIO_TypeDef`, `RCC_TypeDef`), permettant d'accéder directement au matériel en C.

### Bloc 2 : Temporisation Logicielle et Rôle de volatile & __NOP()

```
static void delay(volatile uint32_t time) {
    while (time--) {
        __NOP();
    }
}
```

**Explication :** Le mot-clé `volatile` est obligatoire : il interdit au compilateur d'optimiser et de supprimer la boucle d'attente (*Dead Code Elimination*). L'instruction intrinsèque `__NOP()` génère l'instruction assembleur **NOP** (1 cycle CPU sans modification d'état), garantissant un délai déterministe.

### Bloc 3 : Clock Gating (Activation de l'Horloge GPIOA)

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
```

**Explication :** À la mise sous tension, tous les périphériques sont éteints pour économiser l'énergie. Le port GPIOA est connecté au bus haute performance **AHB1**. Le bit 0 (`RCC_AHB1ENR_GPIOAEN`) est mis à `1` avec l'opérateur `|=` (OU binaire) pour alimenter le GPIOA sans altérer les autres périphériques.

### Bloc 4 : Configuration de la Broche PA5 en Sortie Générale (01)

```
GPIOA->MODER &= ~(3U << (5 * 2));   // 1. Reset des bits 10 et 11 à 00
GPIOA->MODER |=  (1U << (5 * 2));   // 2. Écriture de 01 dans les bits 10 et 11
```

**Explication :** Dans `MODER`, chaque broche utilise 2 bits. Pour PA5, il s'agit des bits [11:10] ($5 \times 2 = 10$).
- Ligne 1 : Le masque `~(3U << 10)` avec `3U = 0b11` remet à zéro les bits [11:10] sans toucher aux autres broches.
- Ligne 2 : `(1U << 10)` inscrit la valeur `01`, configurant la broche PA5 en mode Sortie générale (Output).

### Bloc 5 : Boucle Infinie while(1) et Pilotage du Niveau Logique (ODR)

```
while (1) {
    GPIOA->ODR |= (1U << 5);    // LED ON : PA5 = 3.3V
    delay(1000000);
    GPIOA->ODR &= ~(1U << 5);   // LED OFF : PA5 = 0V
    delay(1000000);
}
```

**Explication :** Le bit 5 du registre `ODR` contrôle la tension de sortie de PA5.
- `ODR |= (1U << 5)` met le bit 5 à 1 : tension = 3.3V, le courant circule et la LED s'allume.
- `ODR &= ~(1U << 5)` met le bit 5 à 0 : tension = 0V, le courant est nul et la LED s'éteint.

## IV Compilation, Résolution de l'Erreur Keil et Flashage (LOAD)

### 1. Choix et Configuration du Compilateur (Version 6 vs Version 5)

Dans Keil µVision, le choix de la version du compilateur s'effectue dans la boîte de dialogue **Options for Target (Alt + F7) → Onglet Target** :

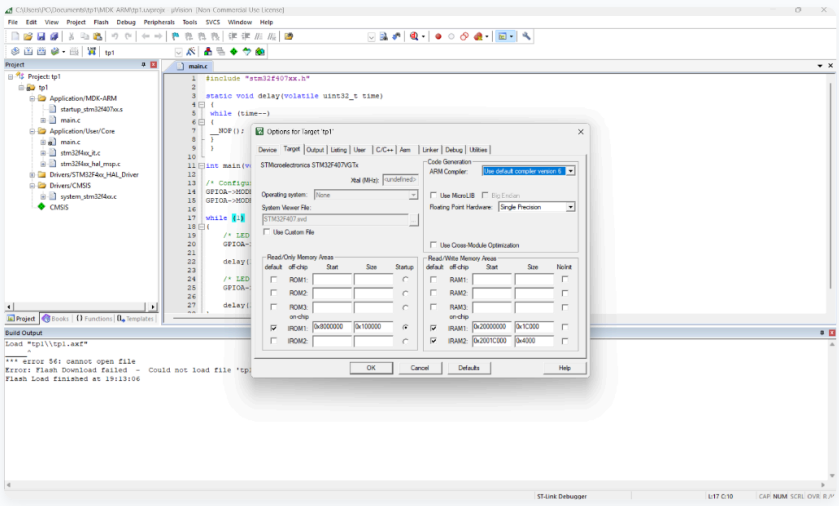


*Figure 3 : Configuration du compilateur dans Options for Target → Target : sélection de "Use default compiler version 6".*

• **Arm Compiler Version 6 (ArmClang) :** Compilateur moderne basé sur la technologie LLVM/Clang. Il offre des optimisations plus rapides et un support complet des standards récents C99/C11.
• **Arm Compiler Version 5 (Armcc) :** Compilateur historique de Keil (legacy), offrant une compatibilité maximale avec les anciens fichiers assembleurs (`armasm`).

> **Explication de l'Erreur : "*** error 56: cannot open file tp1.axf"**
> Cette erreur visible dans la console survient lorsque l'on clique sur le bouton **LOAD (F8)** alors que le projet n'a pas encore été compilé avec succès avec **Build (F7)**. Keil cherche le fichier exécutable binaire `tp1.axf` en mémoire Flash mais ne le trouve pas sur le disque.
> **Règle d'or :** Il faut **toujours compiler d'abord avec F7 (Build)** jusqu'à obtenir `0 Error(s), 0 Warning(s)` **avant** d'appuyer sur **F8 (LOAD)**.

### 2. Résolution en cas de conflit avec Version 6 (--vfemode=force)

Si l'utilisation de la **Version 6** génère l'erreur `Fatal error: L3900U: Unrecognized option '--vfemode=force'`, cela indique que le linker utilisé est issu de la version 5 : il suffit alors de basculer la ligne *ARM Compiler* sur **`Use default compiler version 5`**, de valider avec **OK**, puis de recompiler avec `F7`.

### 3. Procédure pour Téléverser (LOAD F8) sur la Carte

1. **Connecter la Carte au PC**
   Brancher le câble mini-USB sur le port ST-LINK de la carte STM32F4-Discovery.
2. **Configurer le Débogueur (Alt + F7 → Debug)**
   Sélectionner **ST-Link Debugger** à droite. Cliquer sur **Settings** et vérifier que la cible est détectée dans *SW Device*.
3. **Activer "Reset and Run" dans Flash Download**
   Dans l'onglet *Flash Download*, **cocher la case "Reset and Run"** pour que le programme démarre automatiquement après le flashage.
4. **Flasher avec le Bouton LOAD (F8)**
   Appuyer sur `F8` (LOAD) : Keil efface, programme et vérifie la mémoire Flash en 2 secondes.

## V Exécution en Boucle (Loop) et Test par le Curseur

Une fois le code flashé, il existe deux modes de test sur la carte physique :

### 1. Exécution Continue en Boucle Infinie (while(1) Loop)

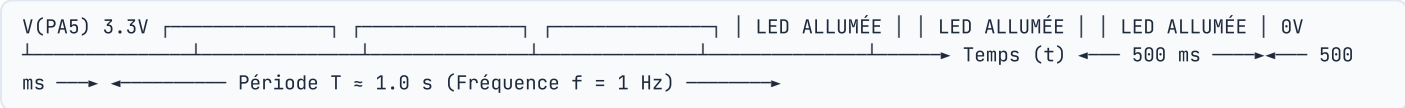
En exécution normale, le microcontrôleur boucle indéfiniment dans `while(1)`. Les instructions allument la LED, attendent 500 ms, l'éteignent et attendent 500 ms. La LED clignote continuellement à la fréquence de 1 Hz sans interruption.

### 2. Débogage et Pilotage Manuel par le Curseur (Run to Cursor / Step Over)

Pour tester et forcer manuellement l'état de la LED depuis Keil :

1. **Lancer la Session de Débogage**
   Appuyer sur `Ctrl + F5` : l'écran bascule en mode debug et le pointeur jaune s'arrête au début de `main()`.
2. **Tester par le Curseur (Run to Cursor Line — Ctrl + F10)**
   Dans l'éditeur, cliquer avec la souris pour placer le curseur de texte sur la ligne `GPIOA->ODR |= (1U << 5);`. Faire un clic droit et choisir **Run to Cursor Line** (ou `Ctrl + F10`). Le processeur exécute immédiatement tout le code précédent et s'arrête pile sur cette instruction.
3. **Avancer Pas-à-Pas (Step Over — F10)**
   Appuyer sur la touche `F10` : la ligne est franchie et **la LED externe sur la breadboard s'allume en direct** !
   En avançant jusqu'à la ligne `GPIOA->ODR &= ~(1U << 5);` et en appuyant sur `F10`, **la LED s'éteint instantanément**.
4. **Reprendre la Boucle Libre (Run — F5)**
   Appuyer sur `F5` pour relancer la boucle infinie libre.

### 3. Résultats Expérimentaux & Chronogramme PA5(t)




Tensions mesurées : $V_{PA5}(\text{Haut}) = 3.28\text{ V}$ (LED On), $V_{PA5}(\text{Bas}) = 0.00\text{ V}$ (LED Off).

## VI Questions d'Approfondissement & Analyse Critique

**Q1 : Pourquoi ne doit-on pas écrire directement `GPIOA->MODER = (1 << 10);` ?**

*Réponse :* L'affectation directe avec l'opérateur = écrase la totalité des 32 bits du registre `MODER`. Toutes les autres broches du port A (y compris celles utilisées pour le débogage SWD comme PA13/PA14) seraient immédiatement remises à zéro (mode entrée 00), ce qui couperait la liaison de débogage ST-LINK. L'utilisation du masque combiné `&= ~` et `|=` garantit de ne modifier que les bits 10 et 11 ciblés.

**Q2 : Que se passe-t-il si la ligne `RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;` est omise ?**

*Réponse :* Le module GPIOA n'est pas alimenté par le signal d'horloge du bus AHB1. Tout accès en lecture/écriture à ses adresses mémoires est bloqué par la matrice de bus. Cela déclenche une exception matérielle **HardFault** ou les écritures sont ignorées : la broche reste inactive et la LED ne s'allume jamais.

## VII Conclusion Générale

Ce second travail pratique a permis de maîtriser les compétences clés du développement sur **STM32F407 (ARM Cortex-M4)** :

- Création et paramétrage d'un projet propre sous **Keil MDK-ARM µVision 5** avec CMSIS CORE et Device Startup.
- Compréhension et programmation **Bare-Metal** des registres `RCC->AHB1ENR`, `GPIOA->MODER` et `GPIOA->ODR`.
- Maîtrise du compilateur **Arm Compiler 5** et résolution de l'erreur `--vfemode=force`.
- Téléversement en mémoire Flash via le bouton **LOAD (F8)** avec l'option *Reset and Run*.
- Débogage et test matériel précis à l'aide des fonctions **Run to Cursor (Ctrl + F10)** et **Step Over (F10)**.

---

**Institut Supérieur d'Informatique et de Multimédia de Gabès**
Filière : Licence en Informatique des Systèmes Industriels (LISI 3)
Année Universitaire : 2026 - 2027

**Membres du Groupe B :**
Mohamed Amine Bouguila • Anis • Yousri